

SKILL 21.22.23.24

21.To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

1. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

INPUT:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main() {
    FILE *fp;

    fp = fopen("missing.txt", "r");

    if (fp == NULL) {
        fprintf(stderr, "Runtime Error: %s\n",
                strerror(errno));

        FILE *log = fopen("error.log", "a");

        if (log != NULL) {
            fprintf(log,
                    "Error: Unable to open missing.txt\n");
            fclose(log);
        }

        printf("Program recovered gracefully.\n");

        return 0;
    }

    fclose(fp);

    return 0;
}
```

output:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill21.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill21.c -o skill21
sushmitha@LAPTOP-B907B06L:~$ ./skill21
Runtime Error: No such file or directory
Program recovered gracefully.
sushmitha@LAPTOP-B907B06L:~$ |
```

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

2. INPUT:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *numbers;

    numbers = malloc(5 * sizeof(int));

    if (numbers == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        numbers[i] = i + 1;

    printf("Numbers:\n");

    for (int i = 0; i < 5; i++)
        printf("%d ", numbers[i]);

    printf("\n");

    free(numbers);

    printf("Memory released successfully.\n");

    return 0;
}
```

output:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill22.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill22.c -o skill22
sushmitha@LAPTOP-B907B06L:~$ ./skill22
Numbers:
1 2 3 4 5
Memory released successfully.
sushmitha@LAPTOP-B907B06L:~$ |
```

23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

3. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documenta

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    pid_t p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);
```

GNU nano 8.7.1

```
        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    pid_t p3 = fork();

    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Pipeline completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill23.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill23.c -o skill23
sushmitha@LAPTOP-B907B06L:~$ ./skill23
48
Pipeline completed successfully.
sushmitha@LAPTOP-B907B06L:~$ |
```

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

INPUT:

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("=====\n");
    printf("        PROJECT ORGANIZATION\n");
    printf("=====\n");

    printf("1. Organize Source Code\n");
    printf("2. Improve Build Scripts\n");
    printf("3. Add README\n");
    printf("4. Verify Repository Structure\n");
    printf("5. Tag Releases\n");
    printf("6. Validate Deliverables\n");

    printf("\n=====\n");
    printf("        DEMO SCENARIOS\n");
    printf("=====\n");

    printf("1. Create Demo Scenarios\n");
    printf("2. Validate Features\n");
    printf("3. Rehearse Presentation\n");
    printf("4. Demonstrate Pipelines\n");
    printf("5. Demonstrate Signals\n");
    printf("6. Final Project Review\n");

    printf("\nProject preparation completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill24.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill24.c -o skill24
sushmitha@LAPTOP-B907B06L:~$ ./skill24
```

```
=====
PROJECT ORGANIZATION
```

- ```
=====
1. Organize Source Code
2. Improve Build Scripts
3. Add README
4. Verify Repository Structure
5. Tag Releases
6. Validate Deliverables
```

```
=====
DEMO SCENARIOS
```

- ```
=====
1. Create Demo Scenarios
2. Validate Features
3. Rehearse Presentation
4. Demonstrate Pipelines
5. Demonstrate Signals
6. Final Project Review
```

```
Project preparation completed successfully.
```

```
sushmitha@LAPTOP-B907B06L:~$ |
```